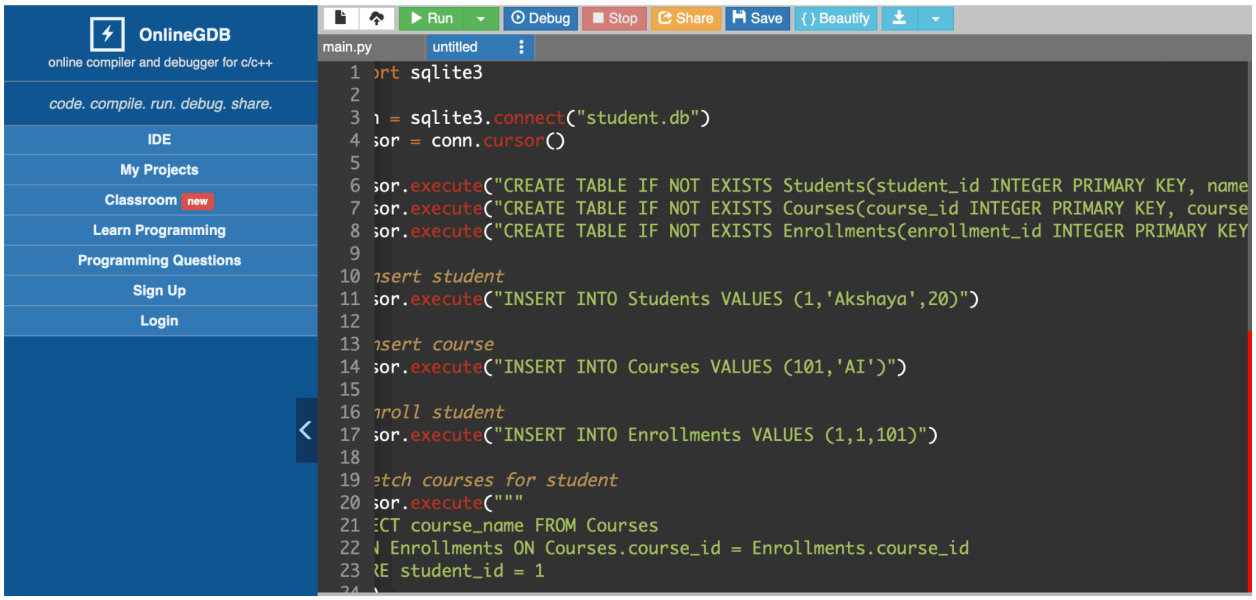
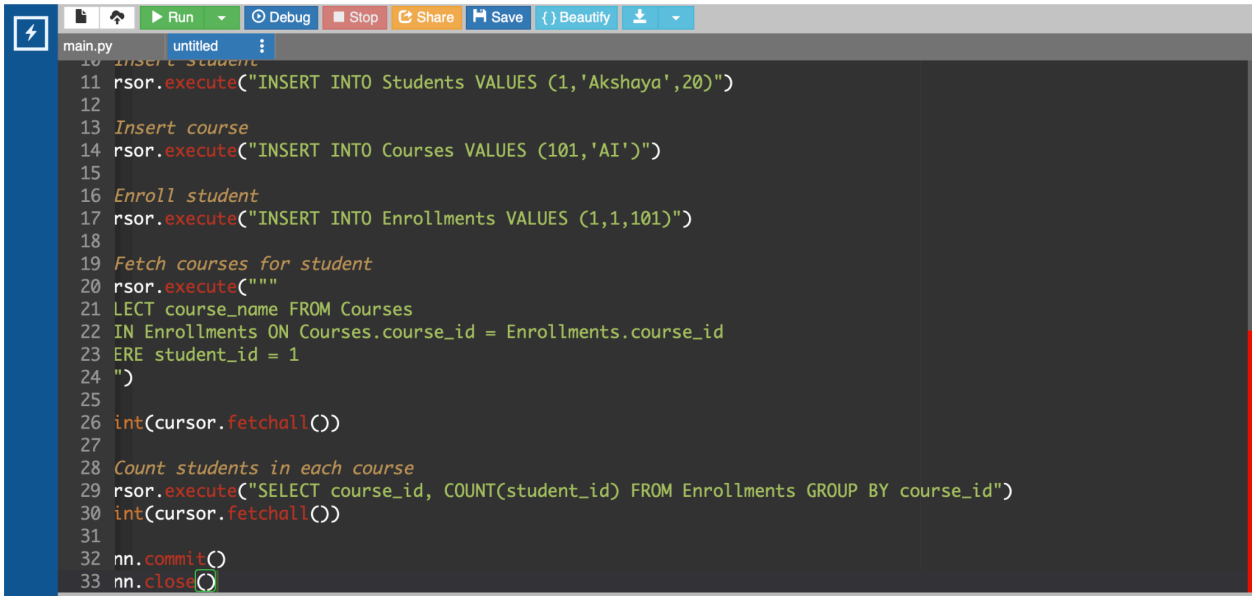
2303A52416
batch=35
assignment=16.1
Task 1 – Student Information System Schema
Task:
Design a database schema for a Student Information System and generate queries using AI.
Instructions:
• Tables: Students, Courses, Enrollments.
• Define primary keys, foreign keys, and relationships.
• Generate queries:
o Insert a new student record.
o Fetch all courses enrolled by a student.
o Count number of students in each course.
Expected Output:
• SQL CREATE TABLE statements and queries for student–course relationships.





Aim

To design a **Student Information System database** using Python and SQLite to store student, course, and enrollment information.

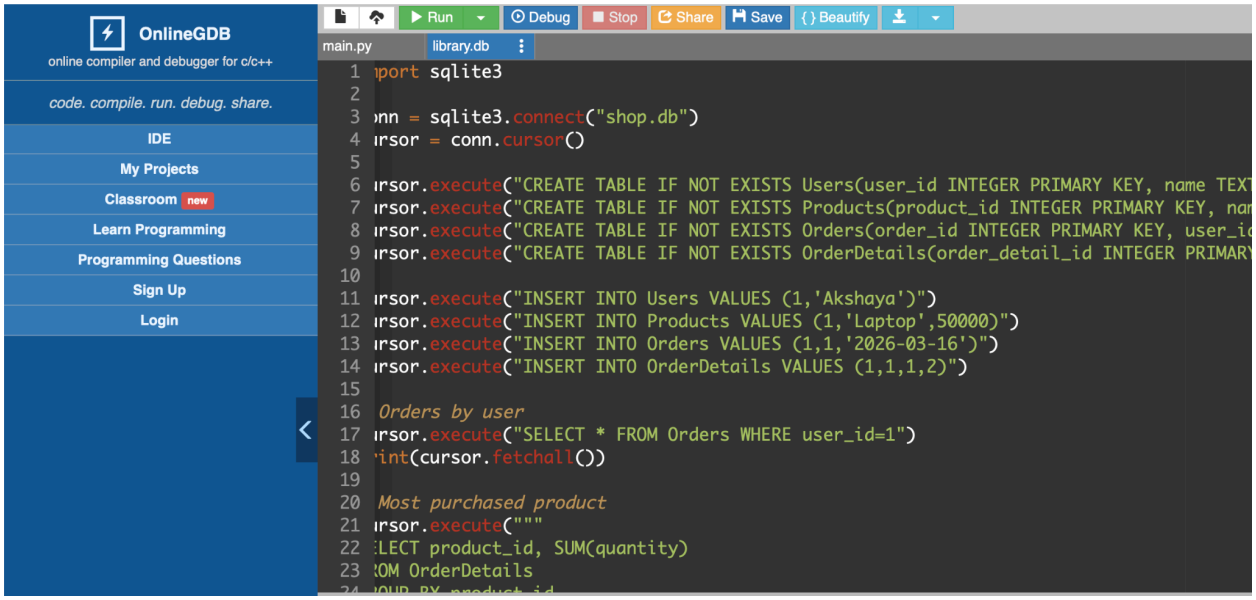
Explanation

1. The program uses the **sqlite3 module** to connect Python with an SQLite database.
2. A database named **student.db** is created.
3. Three tables are created:
    - **Students** → stores student details like ID, name, and age.

- **Courses** → stores course information.
- **Enrollments** → connects students with courses.

4. The **CREATE TABLE** command creates tables if they do not already exist.
5. The program inserts sample data using **INSERT INTO** statements.
6. A **JOIN query** is used to retrieve courses enrolled by a specific student.
7. A **GROUP BY query** counts the number of students in each course.
8. Finally, `commit()` saves the changes and `close()` closes the database connection.

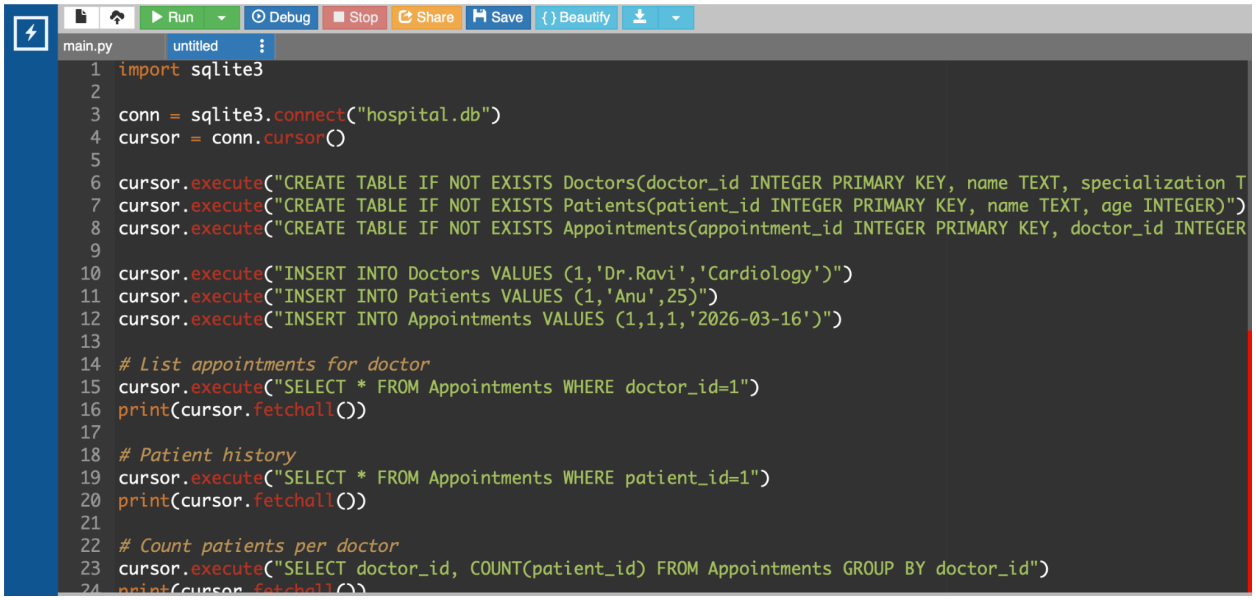
Task 2 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
o List all appointments for a specific doctor.
o Retrieve patient history by patient ID.
o Count total patients treated by each doctor.
Expected Output:
• Normalized schema and SQL queries with joins.

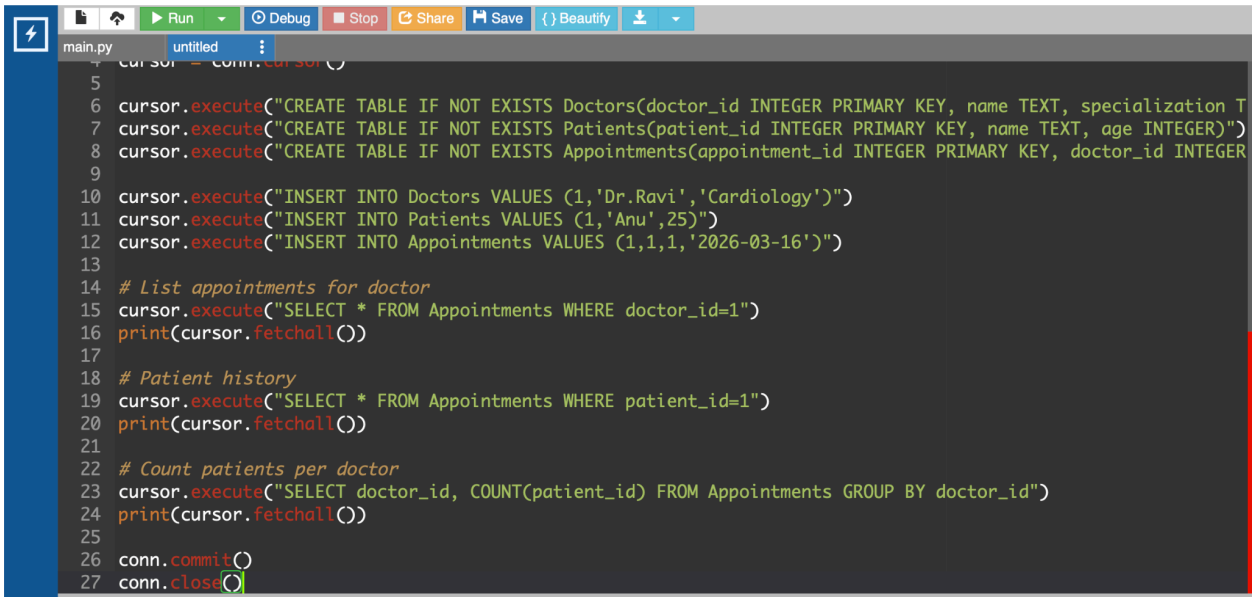


Explanation

1. The program creates a database called **library.db**.
2. Three tables are created:
   - **Books** → stores book ID, title, and author.
   - **Members** → stores member information.
   - **Loans** → records borrowed books and loan dates.

3. Data is inserted into all tables.
4. A **JOIN query** retrieves the titles of books currently issued.
5. The query connects the **Books** and **Loans** tables using the book ID.
6. Another query counts how many books each member has borrowed using **GROUP BY**.
7. The results are printed on the screen.





Explanation

1. The program connects to a database named **hospital.db**.
2. Three tables are created:

    - **Doctors** → stores doctor ID, name, and specialization.
    - **Patients** → stores patient details.
    - **Appointments** → records appointment details including doctor, patient, and date.
3. Sample records are inserted into each table.
4. SQL queries are executed using the **execute() function**.
5. The query **SELECT * FROM Appointments WHERE doctor_id=1** retrieves all appointments for a specific doctor.
6. Another query retrieves patient appointment history.
7. A **GROUP BY query** counts the number of patients treated by each doctor.